

Advanced Programming - Exam Overview

Brent van Bladel



Evaluation

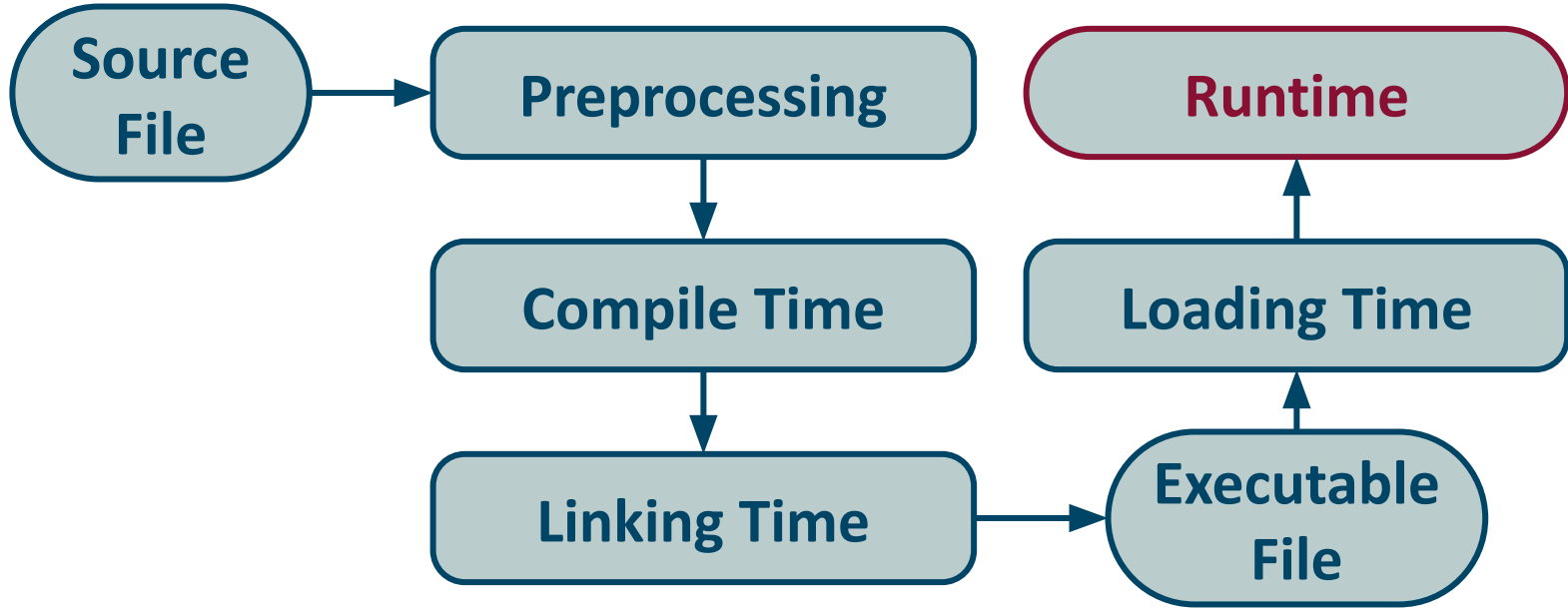
- 50% Theory Exam
 - Questions about code snippets
- 50% Project
 - More information during the practical sessions

**You need to pass each part in order
to pass the course!**

Chapters - Overview

- Chapter 1: Basic Concepts
 - Compiler timings
 - Declaration vs Definition

C++ Timings



Declarations & Definitions

A **declaration** introduces an identifier and binds its static type.

It is what the compiler needs to accept references to that identifier.

A **definition** instantiates/implements an identifier.

It's what the linker needs in order to link references to those entities.

```
1.  extern int bar;  
2.  extern int g(int, int);  
3.  double f(int, double);  
4.  class Foo;
```

```
1.  int bar;  
2.  int g(int a, int b) {  
3.      return a + b; }  
4.  double f(int a, double b) {  
5.      return a * b; }  
6.  class Foo {  
7.      };
```

Note that a definition can also be a declaration.

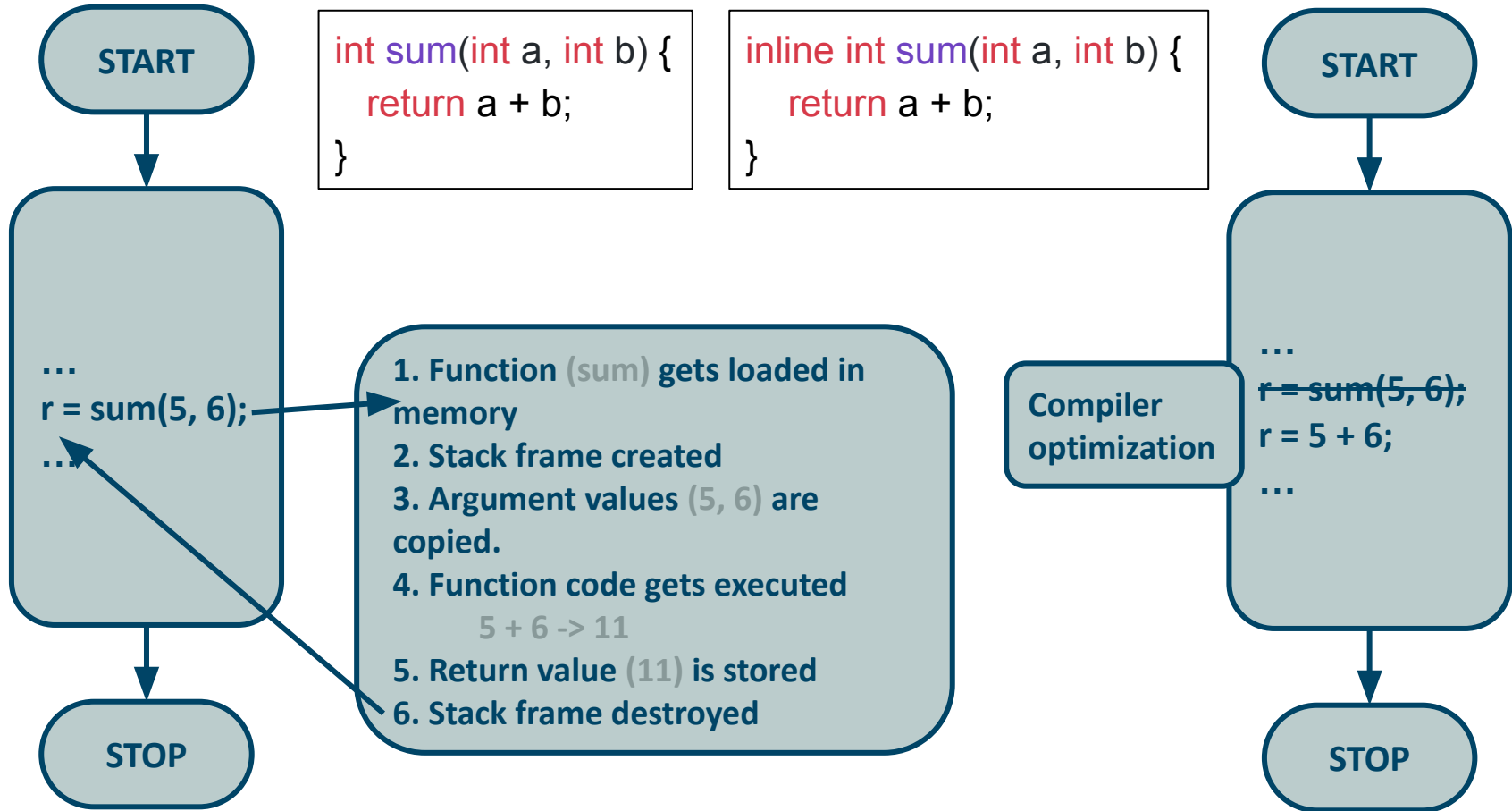
Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
 - Inline
 - constexpr

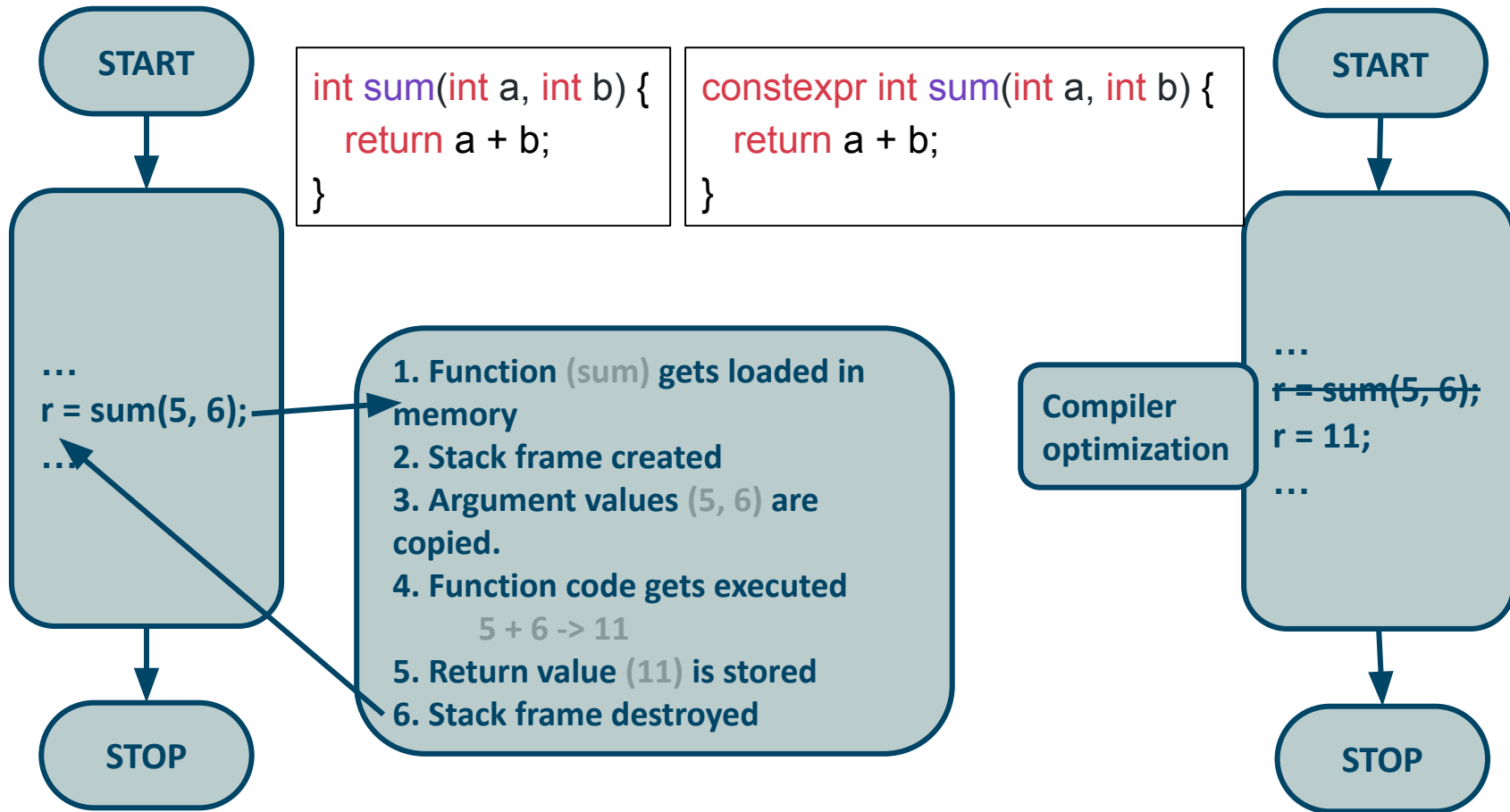
Standard Function

vs

Inline Function



Standard Function vs Cxxpr Function



Chapters - Overview

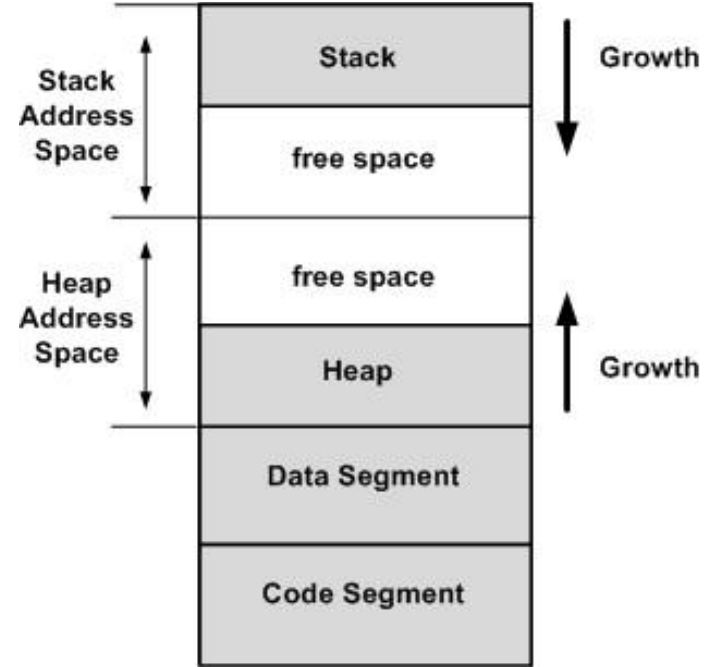
- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
 - Stack vs Heap
 - Static Memory

Stack vs Heap

Stack is used for local variables, function arguments, and control information, such as return addresses.

The **heap** is a flexible area for storing objects with dynamic lifetimes.

Note that allocating on the heap is more expensive compared to the stack!

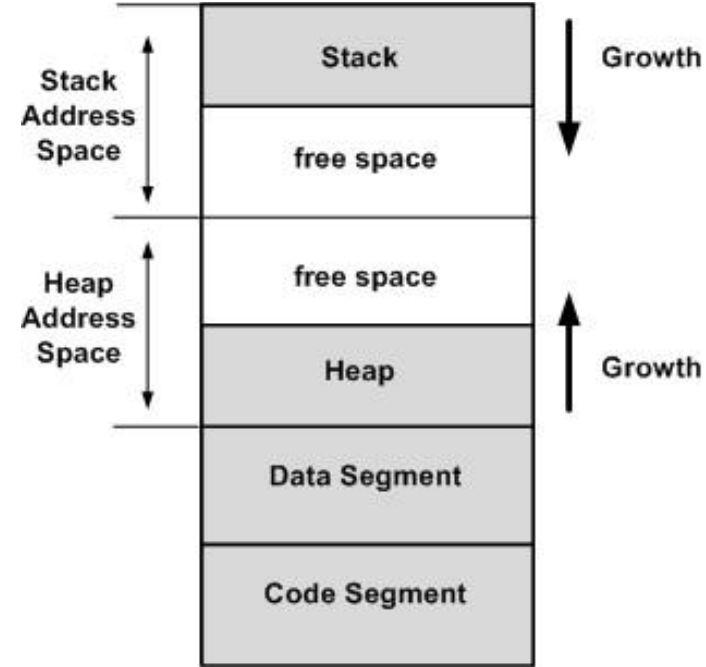


Static Memory Area

The static area of memory is of fixed size.

It consists of 2 parts:

- **Read-only code:** the machine language program and constants, including string constants.
- **Static data:** global variables, which can be changed by the program, but always occupy a fixed number of bytes.



Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
 - Copy Constructor vs Assignment Operator
 - Destructors

Copy Constructor vs Assignment Operator

A copy constructor constructs and initializes memory.

An assignment operator assigns a state to a previously constructed object.

```
1.  class Coordinate {  
2.      float _x;  
3.      float _y;  
4.      public:  
5.          Coordinate(float x, float y) : _x{x}, _y{y} {};  
6.          Coordinate(const Coordinate& other) : _x{other._x}, _y{other._y} {};  
7.          Coordinate& operator=(const Coordinate& other) {  
8.              _x = other._x;  
9.              _y = other._y;  
10.             return *this;  
11.         }  
12.         ~Coordinate() = default;  
13.     };
```

Destructors

A **destructor** is called explicitly using *delete* for objects on the heap and implicitly at the end of scope for objects on the stack.

```
1. int main() {  
2.     Coordinate* c_on_heap = new Coordinate{5, 10};  
3.     Coordinate c_on_stack{5, 10};  
4.  
5.     delete c_on_heap; // destructor called on heap  
6.  
7. } // end of scope: destructor called on stack
```

Chapters - Overview

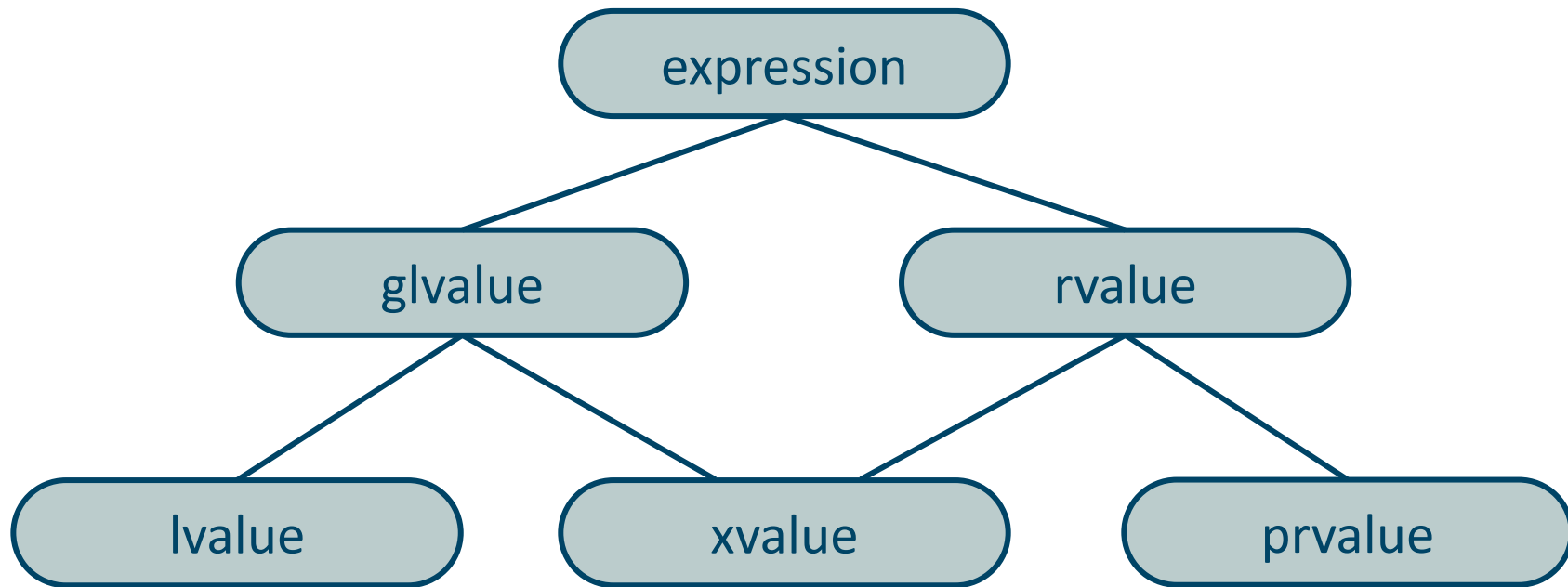
- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
 - Ownership
 - C++ Value Categories

Ownership

A unit of code has **ownership** of a created object if it has the authority on the lifetime of that object.

The owner of an object is responsible for clean-up of the object.

C++ Value Categories



Every expression is an lvalue, an xvalue, or an prvalue.

Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
- Chapter 6: Class Design
 - Class Interface - ADT

Class Interface - ADT

A well-designed class interface should form an **Abstract Data Type**.

The class interface promises a certain set of properties and abilities:

- Pre- & post-conditions
- Invariants

Advantages

- **Encapsulation**
The user does not need to know the implementation details.
- **Localization of change**
Changes to the ADT are hidden behind the interface.

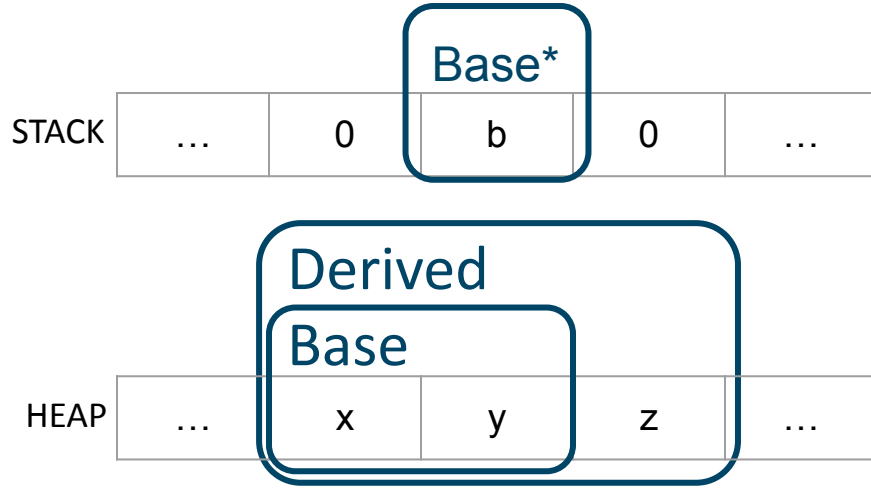
Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
- Chapter 6: Class Design
- Chapter 7: Class Derivation
 - Polymorphism
 - vtable

Polymorphism

Since a derived object contains an object of the base class, a pointer to the base class can point to an object of the derived class.

(both objects start on the same address)



```
1. class Base {
2.     int x;
3.     int y;
4.     public:
5.         virtual void print() {
6.             std::cout << x << y << std::endl; };
7. };
8.
9. class Derived : public Base {
10.     int z;
11.     public:
12.         void print() override {
13.             std::cout << x << y << z << std::endl; };
14. };
15.
16. int main (){
17.     Base* b = new Derived{};
18.     b->print();
19. }
```

vtable

Construction happens from Base (Foo) to Derived (Bar).

1) Foo constructor places the virtual function print in the vtable:

Foo									
...	0	Foo::print()	x	y	0	0	0	0	...

2) Bar constructor overrides the virtual function, and thus updates the vtable.

Bar									
Foo									
...	0	Bar::print()	x	y	z	0	0	0	...

3) A function call to a virtual function will check the vtable.

Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
- Chapter 6: Class Design
- Chapter 7: Class Derivation
- Chapter 8: Design Patterns
 - Design Principles

Design Principles

- Liskov substitution principle
- Type Conformity
- Principle of Closed Behavior
- Single-responsibility principle
- Open–closed principle
- Low coupling / high cohesion

Object-oriented designs following these principles are more understandable, flexible, and maintainable.

Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
- Chapter 6: Class Design
- Chapter 7: Class Derivation
- Chapter 8: Design Patterns
- Chapter 9: Multi-threading
 - Lambda Functions - Function Pointer
 - Data Race

Lambda Functions - Function Pointer

```
1.  vector<int> v{1,2,3,4,5};
2.
3.  bool (*is_even)(int) = [](int i) -> bool {return i % 2 == 0;};
4.
5.  if (all_of(v.begin(), v.end(), is_even))
6.      cout << "All numbers are even";
7.
8.  if (any_of(v.begin(), v.end(), is_even))
9.      cout << "Some numbers are even";
10.
11. if (none_of(v.begin(), v.end(), is_even))
12.     cout << "All numbers are odd";
```

Lambda functions can also be stored as a function pointer*.

*if it does not capture anything

Data Race

A **data race** occurs when 2 instructions from different threads access the same memory location without synchronization, and at least one of the accesses is a write.

```
1.  int main() {  
2.      int count = 0;  
3.      std::thread t1{ [&]() { count++; } };  
4.      std::thread t2{ [&]() { count++; } };  
5.      t1.join();  
6.      t2.join();  
7.      std::cout << count << std::endl;  
8.  }
```

Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
- Chapter 6: Class Design
- Chapter 7: Class Derivation
- Chapter 8: Design Patterns
- Chapter 9: Multi-threading
- Chapter 10: Generic Programming
 - Templates
 - Compile-time polymorphism

Templates

```
1. int sum(int a, int b){  
2.     return a + b;  
3. }  
4.  
5. double sum(double a, double b){  
6.     return a + b;  
7. }  
8.  
9. long sum(long a, long b){  
10.     return a + b;  
11. }
```

“concrete algorithms”

Generate*

```
1. template <typename Number>  
2. Number sum(Number a, Number b){  
3.     return a + b;  
4. }
```

“generic algorithms”

*at compile-time

Class Templates & Inheritance

```
1. class IReporter {  
2.     public:  
3.         // compilation error  
4.         template<typename StringType>  
5.         virtual void print(StringType message) = 0;  
6.     };
```

Template functions cannot be virtual.

→ Templated functions provide compile-time polymorphism,
virtual functions provide runtime polymorphism.

(Size of the vtable would not be known until the entire program has been translated.)

Chapters - Overview

- Chapter 1: Basic Concepts
- Chapter 2: Functions & Namespaces
- Chapter 3: Memory
- Chapter 4: Classes & Memory
- Chapter 5: Advanced Memory
- Chapter 6: Class Design
- Chapter 7: Class Derivation
- Chapter 8: Design Patterns
- Chapter 9: Multi-threading
- Chapter 10: Generic Programming

Advanced Programming

Brent van Bladel

